

Enterprise Insurance Claims Policy & Coverage Copilot

Agentic RAG Copilot | FDE Project — Problem Statement, Proposed Solution, Deployment, and Easy-to-Understand Examples

Customer	ShieldWave Insurance (fictional)
Environment	Multi-state home & auto insurer, ~1,200 claims agents / adjusters
Use Case	Internal Claims Coverage & Policy Support
Deployment Target	Deploy the complete production application on DigitalOcean Cloud using Docker services, including the FastAPI backend, UI, application dependencies, environment configuration, and production runtime.

1. Problem Statement

ShieldWave Insurance maintains a large internal claims and policy knowledge base covering coverage terms, exclusions, deductible structures, claims intake procedures, escalation rules, and state-specific filing requirements. Although this information already exists, claims agents and policyholders still contact senior adjusters repeatedly because they cannot quickly identify the correct clause or determine which policy terms apply to a specific loss.

Traditional keyword search often returns multiple loosely related policy sections rather than one clear, clause-level answer. A normal chatbot introduces a more serious risk in this domain: it may confidently state that something is covered — or excluded — even when reliable policy evidence is unavailable, which carries real financial and legal consequences. Some claims questions also depend on current, external, state-regulated information (disaster declarations, extended filing deadlines) that will never live in a static internal policy document.

The core business challenge

Claims agents and policyholders need fast, specific coverage answers, while the company needs those answers to be grounded, clause-cited, and clearly flagged when they rely on external — rather than internally approved — information.

2. Why Simple RAG Is Not Enough

Simple RAG	Problem in the real world
Question → Retrieve → Generate	Retrieved clauses may be weak, unrelated, or only partially relevant, but the model may still state a coverage determination.
Uses only the private KB	It cannot answer well when the question depends on current state regulation or a recent disaster declaration not yet in the policy KB.

No evidence decision	The application does not decide whether the retrieved policy evidence is clause-specific enough before answering — and doesn't flag externally-sourced answers for human review.
----------------------	--

3. Proposed Solution

Build an Enterprise Insurance Claims Policy & Coverage Copilot. The system searches the trusted private policy/claims knowledge base first, evaluates the retrieved evidence for clause-level specificity, and only generates a coverage answer when the evidence is strong. If internal evidence is weak — commonly true for state regulation or disaster-declaration questions — LangGraph routes the request to Tavily web search. The web evidence is graded as well; if it remains insufficient, the system rewrites the query (clarifying policy line, state, or peril) and retries rather than blindly answering. Any answer sourced externally is explicitly flagged as requiring adjuster review.

- 1 Policyholder or agent asks a coverage/claims question
- 2 LangGraph routes the request (chitchat vs. claims question)
- 3 Retrieve relevant clauses from the private Pinecone policy knowledge base
- 4 Grade whether the private evidence names a specific coverage/exclusion clause
- 5 GOOD → Generate a grounded answer citing the specific policy clause
- 6 WEAK → Search the web using Tavily for state regulation / disaster declaration context
- 7 Grade the web evidence for specificity and currency
- 8 If needed, rewrite the query (policy line / state / peril) and retry
- 9 Return the answer, sources, decision trace, and adjuster-review flag if externally sourced

4. Easy Example #1 — Answer Found in the Company Policy KB

Policyholder: "Is water damage from a burst pipe covered under my homeowner policy?"

The sudden-discharge coverage clause already exists in ShieldWave's homeowner policy wording. The system retrieves the relevant clause from Pinecone, grades the evidence as clause-specific, and generates the answer from internal policy knowledge — citing Section 3.1 and noting the \$1,000 deductible — without unnecessary web search.

5. Easy Example #2 — Private Knowledge Is Not Enough

Claims agent: "What's the extended claims filing deadline after the recent Texas flooding declaration?"

ShieldWave's internal policy wording has no way to know about a disaster declared last week. The evidence grader identifies this gap, LangGraph routes the request to Tavily, external evidence about the state emergency declaration is evaluated, and the system responds only after obtaining sufficient evidence — clearly flagging the answer as external information requiring adjuster/compliance validation before it is used in an actual claim decision.

6. What Makes This Agentic RAG?

Unlike a fixed Question → Retrieve → Generate pipeline, this system makes decisions during execution: retrieve, grade, route, search, rewrite, retry, and generate only when evidence is sufficient — and flag externally-sourced answers for human sign-off.

Normal RAG	Agentic RAG in this project
Retrieve once	Retrieve, grade, and retry when necessary
Fixed flow	Conditional LangGraph routing
Generate after retrieval	Generate only after evidence evaluation
Private KB only	Private policy KB first, controlled regulatory web fallback when needed
No compliance flag	Externally-sourced answers flagged as requiring adjuster review

7. Proposed Product Components

- **LangGraph:** agentic decision workflow.
- **Pinecone:** private embedded policy/claims knowledge (index: shieldwave-claims-rag).
- **OpenAI embeddings:** vector representations for documents and queries.
- **OpenAI LLM:** routing, grading, query rewriting, and grounded generation.
- **Tavily:** controlled web fallback for state regulations and disaster declarations.
- **FastAPI:** production backend APIs.
- **HTML/CSS/JavaScript:** claims-agent-facing assistant interface and workflow trace.
- **SQLite:** audit logging, including the adjuster-review compliance flag.
- **Document ingestion:** authorized claims/compliance staff can add PDF, TXT, Markdown, and DOCX policy knowledge.
- **Docker:** packages the application and its runtime consistently for deployment.
- **DigitalOcean Cloud:** hosts the containerized production application.

8. Expected Business Value

- Reduce repetitive coverage questions escalated to senior adjusters.
- Help agents and policyholders find and understand clause-level coverage faster.
- Prioritize approved private policy knowledge over generic guesses.
- Reduce unsupported coverage determinations through evidence grading.
- Handle regulatory/disaster-declaration knowledge gaps with controlled web fallback.
- Flag externally-sourced answers for mandatory adjuster review — reducing compliance risk.
- Provide source and workflow traces for auditability in a regulated industry.

- Create a deployable, production-style claims support product rather than only a notebook demo.

9. Deployment — DigitalOcean + Docker

The final product will be containerized with Docker and deployed to DigitalOcean Cloud. The deployment demonstrates the Forward Deployed Engineer lifecycle beyond local development: package the FastAPI application and frontend, configure production environment variables and external services, run the container in the cloud, expose the application to users, validate the deployed workflow, and observe the running product.

- 1 Build and test the complete application locally
- 2 Create the Dockerfile and container image
- 3 Configure required environment variables and API credentials
- 4 Provision the DigitalOcean cloud runtime
- 5 Deploy and run the Dockerized FastAPI + UI application
- 6 Connect production services such as Pinecone, OpenAI, and Tavily
- 7 Verify ingestion, Agentic RAG workflow, sources, adjuster-review flags, and audit trace in production
- 8 Observe, troubleshoot, and improve the deployed product

10. FDE Perspective

The goal of the Forward Deployed Engineer is not simply to demonstrate RAG in a notebook. The FDE starts from the customer's claims support problem and delivers a usable product: knowledge ingestion, agentic orchestration, APIs, UI, auditability, testing, Docker packaging, cloud deployment, observation, and continuous improvement.

Customer Problem → Understand Workflow → Connect Policy Knowledge → Build Agentic RAG → Expose APIs → Build UI → Test → Dockerize → Deploy on DigitalOcean → Observe → Improve